

Module 01: Systems — Generative Process vs Generative Model

Learning Objectives

1. Distinguish the **generative process** (the true environment) from the **generative model** (the agent's internal model).
2. Implement a `DiscreteEnvironment` with true A and B matrices and step through it programmatically.
3. Explain how observations are sampled from the true likelihood matrix and how states transition under the true dynamics.

Introduction

Every Active Inference agent lives inside a world it cannot access directly. The **generative process** is the real causal structure of that world — the true states, transition probabilities, and observation likelihoods. The **generative model** is the agent's *approximation* of that structure, encoded in matrices it can update.

This distinction is the starting point for everything in computational Active Inference. If the generative model perfectly matched the generative process, free energy would be zero and the agent would have nothing left to learn. In practice, the mismatch between the two is exactly what drives perception, action, and learning.

In the `active_inference` library, these two sides of the coin are represented by separate classes:

Concept	Class	Matrices
Generative Process	<code>DiscreteEnvironment</code>	<code>true_A</code> , <code>true_B</code> (ground truth)
Generative Model	<code>GenerativeModel</code>	<code>A</code> , <code>B</code> , <code>C</code> , <code>D</code> , <code>E</code> (agent's beliefs)

Key Concepts

1. The Generative Process as Ground Truth

The generative process defines the true causal structure of the environment. In the discrete case, this consists of:

- **True states** $s \in \{0, 1, \dots, N_s - 1\}$: the hidden states of the world.
- **True likelihood** $\mathbf{A}_{\text{true}}$: a matrix where $A[o, s] = P(o \mid s)$ — the probability of observing o when the world is in state s .
- **True transitions** $\mathbf{B}_{\text{true}}$: a tensor where $B[s', s, a] = P(s' \mid s, a)$ — the probability of transitioning to state s' given the current state s and action a .

In code, you create a `DiscreteEnvironment` by supplying these matrices:

```
import numpy as np
from active_inference.agent import DiscreteEnvironment

# True likelihood: observation 0 is strong evidence for state 0
true_A = np.array([[0.9, 0.1],
                   [0.1, 0.9]])

# True transitions: action 0 = stay, action 1 = swap
true_B = np.zeros((2, 2, 2))
true_B[:, :, 0] = np.eye(2)          # stay
true_B[:, :, 1] = np.array([[0, 1],
                             [1, 0]]) # swap

env = DiscreteEnvironment(true_A, true_B, initial_state=0)
```

2. Observation Generation

When the environment is queried, it samples an observation from the true likelihood column corresponding to the current hidden state:

$$s_{o_t} \sim \text{Cat}(\mathbf{A}_{\{\text{true}\}}[\cdot, s_t])$$

This is implemented via `env.step(action)` , which:

1. Transitions the hidden state: $s_{t+1} \sim \text{Cat}(\mathbf{B}_{\{\text{true}\}}[\cdot, s_t, a_t])$
2. Generates an observation: $s_{o_{t+1}} \sim \text{Cat}(\mathbf{A}_{\{\text{true}\}}[\cdot, s])$
3. Returns the observation index

```
obs = env.reset(initial_state=0)    # sample first observation
obs = env.step(action=1)            # take swap action, get new observation
print(f"State: {env.state}, Obs: {obs}, Timestep: {env.timestep}")
```

3. State-Space Dimensionality

The environment's dimensionality is derived from the matrices:

Property	Derivation	Example
<code>num_obs</code>	Rows of A	2
<code>num_states</code>	Columns of A	2
<code>num_actions</code>	Third dimension of B (or 1 if B is 2-D)	2

A 2-D B matrix is treated as a single-action environment (the agent has no choice).

4. History Tracking

The environment records a complete trajectory:

```

env = DiscreteEnvironment(true_A, true_B, initial_state=0)
env.reset(initial_state=0)
for a in [0, 1, 1, 0]:
    env.step(a)

print(env.history["states"])          # [0, 0, 1, 0, 0]   (initial + 4 steps)
print(env.history["observations"])    # [obs0, obs1, obs2, obs3]
print(env.history["actions"])         # [0, 1, 1, 0]

```

This history can be visualized with `plot_environment_trajectory()`.

5. The Generative Model as the Agent's Hypothesis

While the environment uses `true_A` and `true_B`, the agent constructs its own hypothesis — a `GenerativeModel` — with matrices that may or may not match reality:

```

from active_inference.agent import GenerativeModel

# Agent's model (might differ from truth)
model = GenerativeModel(
    A=true_A.copy(),          # agent's likelihood beliefs
    B=true_B.copy(),          # agent's transition beliefs
    C=np.zeros(2),            # preferences (log scale)
    D=np.array([0.5, 0.5])    # prior over initial state
)

```

The gap between `model.A` and `true_A` (and similarly for B) is what the agent must close through perception and learning. This is the fundamental asymmetry of Active Inference.

Applications

- **Sensory noise:** A `true_A` with off-diagonal probability > 0 simulates noisy observations. Setting `true_A = np.eye(N)` creates a fully observable environment.
- **Stochastic dynamics:** A `true_B` that is not a permutation matrix creates environments where actions have uncertain effects (e.g., slippery gridworlds).
- **Benchmarking:** By comparing the agent's learned A against `true_A`, you can measure how accurately the agent has recovered the environment's causal structure.

Conclusion

The generative process / generative model distinction structures every Active Inference computation. The `DiscreteEnvironment` class encodes this process, providing the ground truth that agents must infer. In Module 02, we build the agent class that maintains and updates a `GenerativeModel` to minimize the mismatch.